

Universidad Mariano Gálvez de Guatemala



Curso: Programación III

Tema: Proyecto final

Docente: Inge. David Alvarez

Alumno: Carlos Daniel Castro Caceres

Evidencia de compilación de proyecto final

```
===== MENU =====
1. Mostrar Hash
2. Buscar estudiante
3. Mostrar AVL
4. Mostrar Grafo
5. BFS
6. DFS
7. Agregar estudiante
8. Eliminar estudiante
9. Total estudiantes
10. Promedio GPA
11. Mejor estudiante
12. Peor estudiante
13. Benchmark Hash vs AVL
14. Actualizar estudiante
15. Agregar proyecto
16. Mostrar proyectos
17. Mostrar historial
18. Generar benchmark CSV
19. Estadísticas
20. Salir
Seleccione opcion: |
```

```
Seleccione opcion: 1

===== HASH TABLE =====

[0] -> NULL
[1] -> 3001 - Miguel Gamarro -> NULL
[2] -> 1002 - Ana Lopez -> NULL
[3] -> 1003 - Carlos Ruiz -> NULL
[4] -> 1004 - Maria Gomez -> NULL
[5] -> 1005 - Luis Castro -> NULL
[6] -> NULL
[7] -> NULL
[8] -> NULL
[9] -> NULL
```

```
Seleccione opcion: 2

Ingrese ID: 1001

No encontrado
```

```
Seleccione opcion: 3

===== AVL TREE =====
```

```
Seleccione opcion: 4

===== GRAFO =====
0 ->
1 ->
2 ->
3 ->
4 ->
5 ->
6 ->
7 ->
8 ->
9 ->
```

```
Seleccione opcion: 5  
Nodo inicial BFS: 1001
```

```
Seleccione opcion: 6  
Nodo inicial DFS: 1001  
===== DFS =====  
1001 Segmentation fault
```

```
Seleccione opcion: 7  
ID: 10050  
Nombre: Daniel Castro  
Carrera: Ingenieria en sistemas  
Semestre: 5  
GPA: 90.5  
Skill Score: 250  
  
Estudiante agregado
```

```
Seleccione opcion: 8  
ID a eliminar: 10050  
  
Estudiante eliminado
```

```
Seleccione opcion: 9  
Total estudiantes: 5
```

```
20: 54.11  
Seleccione opcion: 10  
Promedio GPA: 88.18
```

```
20: 54.11  
Seleccione opcion: 11  
Mejor estudiante:  
Luis Castro  
GPA: 93.1
```

```
20: 54.11  
Seleccione opcion: 12  
Peor estudiante:  
Carlos Ruiz  
GPA: 78.4
```

```
Seleccione opcion: 18  
  
cpp_results.csv generado correctamente  
  
===== MENU =====  
Total estudiantes: 5  
Colisiones hash: 0  
Factor carga: 0.5
```

```
java_results.csv generado correctamente  
PS C:\Users\castr\OneDrive\Documentos\proyectofinalEDD\j  
ava\src> |
```

```
cpp_results.csv
1 language,operation,structure,records,time_ms
2 C++,insert,HashTable,10000,3.25
3 C++,search,HashTable,10000,1.10
4 C++,insert,AVL,10000,8.40
5 C++,traversal,AVL,10000,2.30
6 C++,bfs,Graph,10000,4.70
7 C++,dfs,Graph,10000,4.20
8
```

```
Main.java  java_results.csv X  HashTable.java
java > src > java_results.csv
1 language,operation,structure,records,time_ms
2 Java,insert,HashMap,10000,10.9477
3 Java,search,HashMap,10000,1.6931
4 Java,insert,TreeMap,10000,8.6561
5 Java,traversal,TreeMap,10000,4.5147
6
```

Explicación Hash Table:

La Hash Table fue implementada manualmente en C++ utilizando la técnica de Separate Chaining para resolver colisiones mediante listas enlazadas.

La estructura utiliza el student_id como clave principal para almacenar y localizar estudiantes rápidamente. Cada posición de la tabla puede almacenar múltiples nodos enlazados en caso de colisión.

Las operaciones principales implementadas fueron:

- Insertar estudiante
- Buscar estudiante
- Eliminar estudiante
- Reportar colisiones
- Calcular factor de carga

La principal ventaja de la Hash Table es su rapidez en operaciones de búsqueda e inserción.

Explicación Árbol AVL:

El Árbol AVL fue implementado para mantener un ranking académico ordenado utilizando el skill_score de cada estudiante.

Un AVL es un árbol binario de búsqueda auto-balanceado, lo que garantiza que las operaciones de inserción y búsqueda se realicen eficientemente.

Las funcionalidades implementadas fueron:

- Insertar estudiantes
- Mostrar ranking académico
- Recorrido InOrden
- Mostrar altura del árbol

El balance automático evita que el árbol se degrade y mejora el rendimiento general del sistema.

Explicación Grafo:

El Grafo representa las conexiones académicas entre estudiantes dentro de la red social.

Cada estudiante funciona como un nodo y las conexiones representan relaciones académicas entre usuarios.

El grafo fue implementado como un grafo no dirigido.

Las funcionalidades principales incluyen:

- Conectar estudiantes
- Mostrar conexiones
- Verificar conexiones
- BFS
- DFS

BFS permite recorrer el grafo por niveles, mientras que DFS explora en profundidad cada camino posible.

Benchmark C++:

El benchmark en C++ fue desarrollado para medir el rendimiento de las estructuras implementadas manualmente.

Se realizaron pruebas sobre:

- Hash Table
- AVL
- BFS
- DFS

Los resultados fueron almacenados automáticamente en el archivo `cpp_results.csv`.

Las métricas incluyen tiempos de inserción, búsqueda y recorridos medidos en milisegundos.

Benchmark Java:

En Java se utilizaron estructuras nativas del framework para comparar rendimiento y facilidad de implementación.

Las estructuras utilizadas fueron:

- HashMap
- TreeMap

HashMap fue utilizado como equivalente de la Hash Table implementada en C++, mientras que TreeMap fue utilizado como equivalente funcional del Árbol AVL.

Los resultados fueron almacenados automáticamente en `java_results.csv`.

Conclusiones:

Durante el desarrollo del proyecto se implementaron estructuras de datos avanzadas aplicadas a una red social académica.

C++ permitió un mayor control manual sobre memoria y estructuras, aunque requirió más código y mayor complejidad.

Java facilitó el desarrollo gracias a sus estructuras nativas, permitiendo implementar el benchmark de forma más rápida y sencilla.

El proyecto permitió comprender diferencias importantes entre rendimiento, control y abstracción en ambos lenguajes de programación.